PES UNIVERSITY

UE24CS352A: MACHINE LEARNING

Week 4: Model Selection and Comparative Analysis

1. Lab Overview & Objectives

Welcome to the lab on building a complete machine learning pipeline. The goal of this assignment is to provide hands-on experience with model selection and evaluation by implementing hyperparameter tuning and ensemble methods—two critical techniques in applied machine learning.

The provided Jupyter Notebook will be used: `Week4_Lab_Boilerplate.ipynb`. This project is divided into three main parts:

- **Part 1: Manual Implementation.** A Grid Search will be built from the ground up using basic loops to understand its inner workings.
- **Part 2: Scikit-learn Implementation.** Scikit-learn's highly optimized, built-in `GridSearchCV` will then be used to perform the same task, demonstrating the efficiency of modern ML libraries.
- **Part 3: Randomized Search Integration.** `RandomizedSearchCV` will be implemented to compare the execution time and performance trade-offs against exhaustive grid search.

Learning Goals

- Understand and implement a pipeline for scaling, feature selection, and model training.
- Grasp the fundamentals of hyperparameter tuning using exhaustive Grid Search and Randomized Search.
- Analyze the impact of feature scaling on distance-based vs. tree-based models.
- Implement k-fold cross-validation to get robust performance estimates.
- Effectively use scikit-learn's Pipeline, `GridSearchCV`, and `RandomizedSearchCV` tools.
- Evaluate and compare the performance of different classification models using various metrics.
- Analyze and interpret model results to draw meaningful conclusions.

2. Datasets

The boilerplate notebook includes functions to load and preprocess four different datasets. At least two of these datasets must be chosen to run the complete pipeline on.

- **Wine Quality:** Predict whether a red wine is of 'good quality' based on its chemical properties.
- **HR Attrition:** Predict employee attrition based on a variety of work-related and personal factors.
- **Banknote Authentication:** Distinguish between genuine and forged banknotes based on image features.
- **QSAR Biodegradation:** Predict whether a chemical is readily biodegradable based on its quantitative structure-activity relationship (QSAR) properties.

3. Key Concepts and The ML Pipeline

Classifiers

Two fundamental classification algorithms will be tuned and compared:

1. **Decision Tree:** A model that uses a tree-like structure of decisions to classify data.
2. **k-Nearest Neighbors (kNN):** A non-parametric algorithm that classifies a data point based on the majority class of its 'k' closest neighbors.

The Scikit-Learn Pipeline

The notebook uses a scikit-learn Pipeline to chain together the data processing and modeling steps. This is crucial for preventing data leakage and streamlining the workflow. The pipeline for each classifier consists of three stages:

Scaler -> SelectKBest -> Classifier

1. **Scaler:** Both `StandardScaler` and `MinMaxScaler` will be tested as hyperparameters to analyze their impact on different algorithms.
2. **SelectKBest:** Selects the top 'k' features using a statistical test (f_classif). The number of features, k, is a key hyperparameter to be tuned.
3. **Classifier:** The final modeling step (Decision Tree or kNN).

4. Instructions and Tasks

The main task is to complete the TODO sections within the `Week4_Lab_Boilerplate.ipynb` notebook for at least two datasets.

Part 1: Manual Grid Search Implementation (`run_manual_grid_search` function)

In this section, the code to perform a grid search from scratch must be completed.

1. **Define Parameter Grids:** In the "Models and Parameter Grids" cell, define the hyperparameter grids (`param_grid_dt`, `param_grid_knn`) to be tested. Remember that parameter names must match the pipeline step names (e.g., 'classifier__max_depth'). Include both `StandardScaler()` and `MinMaxScaler()` in the scaler grid.

2. **Implement the Search Loop:** Inside the `run_manual_grid_search` function, the following is needed:

- Generate all possible combinations of hyperparameters from the defined grid.
- For each combination, perform 5-fold stratified cross-validation.
- Inside each fold, build the pipeline, fit it on the training part of the fold, and evaluate it on the validation part.
- Calculate the average ROC AUC score across all 5 folds for that hyperparameter combination.
- Keep track of the combination that yields the highest average score.

3. **Fit the Best Model:** After the search is complete, the function will automatically refit a new pipeline using the best-found parameters on the entire training dataset.

Part 2: Built-in GridSearchCV Implementation (`run_builtin_grid_search` function)

Here, scikit-learn's powerful `GridSearchCV` will be leveraged to automate the process.

1. **Set up GridSearchCV:** Inside the `run_builtin_grid_search` function, for each classifier, the following is needed:

- Create the same three-step Pipeline as in the manual part.
- Instantiate `GridSearchCV` with the pipeline, the parameter grid, `scoring='roc_auc'`, and a 5-fold StratifiedKFold cross-validator.
- Fit the `GridSearchCV` object on the training data.
- Use the time module to record the execution time of the search.

2. **Extract Results:** The function will automatically extract the best estimator (the pipeline with the best-found parameters), the best parameters, execution time, and the best cross-validation score.

Part 3: Randomized Search Integration (`run_randomized_search` function)

Here, the efficiency of randomized search compared to exhaustive grid search will be evaluated.

1. **Set up RandomizedSearchCV:** Inside the function, for each classifier:

- Define a continuous or expanded parameter distribution (e.g., using `scipy.stats.randint`).
- Instantiate `RandomizedSearchCV` with the pipeline, parameter distributions, `n_iter=10`, `scoring='roc_auc'`, and a 5-fold StratifiedKFold cross-validator.
- Use the time module to record the execution time of the fit.

5. Expected Deliverables

Two items are required to be submitted:

1. **Completed Jupyter Notebooks (.ipynb files)**

- Submit a separate, completed notebook for each chosen dataset (minimum of two).

- The notebooks must be fully executed, with all TODO sections completed and all output cells (printouts, plots) visible.
- Ensure the code is clean, well-commented, and runs without errors from top to bottom.

2. **Comprehensive Lab Report (.pdf file)**
   - A formal report that details the work and findings, following the structure outlined below.

6. Report Structure and Guidelines

The report is a critical part of this lab and should demonstrate an understanding of the concepts.

Title Page

- Project Title, Name, Student ID, Course Name, Submission Date.

1. Introduction

- A brief overview of the project's purpose and the tasks performed (hyperparameter tuning, model comparison, scaling impact).

2. Dataset Description

- For each dataset chosen, provide a brief description: number of features, number of instances, and what the target variable represents.

3. Methodology

- Explain the key concepts: Hyperparameter Tuning, Grid Search, Randomized Search, K-Fold Cross-Validation.
- Describe the ML pipeline used (Scaler, SelectKBest, Classifier).
- Describe the process followed for the manual implementation (Part 1), the scikit-learn `GridSearchCV` implementation (Part 2), and the `RandomizedSearchCV` implementation (Part 3).

4. Results and Analysis

- For each dataset, present the findings clearly. Using tables is highly recommended.
- **Performance Tables:** Use tables to display the key performance metrics (Accuracy, Precision, Recall, F1-Score, and ROC AUC) for the best version of each classifier from Parts 1, 2, and 3.
- **Feature Scaling Impact:** Compare the performance of kNN and Decision Trees under both `StandardScaler` and `MinMaxScaler`. Discuss why one algorithm is sensitive to scaling while the other is scale-invariant.
- **Execution Time Comparison:** Compare the execution time and ROC AUC score of `GridSearchCV` versus `RandomizedSearchCV`. Discuss the trade-offs between exhaustive search and randomized approaches.
- **Visualizations:** Include the final ROC Curve plots and Confusion Matrix plots generated in the notebook. Analyze these plots to compare model performance.

- **Best Model:** Analyze which model performed best overall for each dataset and offer a hypothesis as to why.

5. Screenshots

• Attach screenshots of the output of the code.

6. Conclusion

- Summarize key findings.
- Discuss the main takeaways from this lab. What was learned about model selection, scaling impacts, and search efficiency?

Good luck!